

Ciencia de Datos

Aprendizaje supervisado: clasificación, pronóstico y entrenamiento con datos etiquetados

¿Qué es el aprendizaje supervisado?

Un paradigma de Machine Learning donde el modelo **aprende de ejemplos etiquetados**: pares de entrada **x** y salida esperada **y**.

- El modelo aprende a **mapear entradas a salidas** observando ejemplos
- Requiere un conjunto de datos con **etiquetas conocidas**
- El objetivo: **generalizar** el aprendizaje a datos nuevos, no vistos

Tipos de tareas supervisadas

Tarea	Pregunta que responde	Ejemplo
Clasificación	¿A qué categoría pertenece?	¿Es spam o no? · ¿Qué dígito es?
Pronóstico / Regresión	¿Cuánto valdrá?	Precio de una casa · ventas del mes

La diferencia clave: la salida **y** es **discreta** (clasificación) o **continua** (pronóstico).

Clasificación

Algoritmos de clasificación

Veremos dos enfoques complementarios esta semana:

 **Redes Neuronales básicas** — aprenden representaciones complejas mediante capas de neuronas artificiales

 **Naive Bayes** — clasifica usando probabilidad condicional con el supuesto de independencia entre características

Ambos son puntos de partida fundamentales en clasificación supervisada.

Redes Neuronales básicas

Una red neuronal artificial imita (de forma simplificada) el funcionamiento del cerebro:

- **Neurona artificial:** recibe entradas, aplica pesos, activa una función
- **Capas:** entrada → capas ocultas → salida
- **Entrenamiento:** ajuste de pesos mediante retropropagación (*backpropagation*)

```
from sklearn.neural_network import MLPClassifier

modelo = MLPClassifier(hidden_layer_sizes=(64, 32),
                        activation='relu',
                        max_iter=300,
                        random_state=42)
modelo.fit(X_train, y_train)
```

Estructura de una red neuronal

Componente	Función
Capa de entrada	Recibe las características <code>X</code>
Capas ocultas	Transformaciones no lineales sucesivas
Capa de salida	Produce la predicción final
Función de activación	Introduce no linealidad (<code>relu</code> , <code>sigmoid</code> , <code>softmax</code>)
Función de pérdida	Mide el error del modelo durante el entrenamiento

Naive Bayes

Basado en el **Teorema de Bayes**:

La probabilidad de que un dato pertenezca a una clase dado sus atributos es proporcional a la probabilidad de los atributos dada esa clase.

- **"Naive"** (ingenuo): asume independencia entre características
- Simple, rápido y sorprendentemente efectivo en texto y datos categóricos
- Funciona muy bien con **pocos datos de entrenamiento**

```
from sklearn.naive_bayes import GaussianNB
```

```
modelo_nb = GaussianNB()  
modelo_nb.fit(X_train, y_train)  
y_pred = modelo_nb.predict(X_test)
```


Naive Bayes — variantes

Variante	Cuándo usar
GaussianNB	Características continuas con distribución normal
MultinomialNB	Conteos de frecuencias (ej. bolsa de palabras)
BernoulliNB	Características binarias (presencia/ausencia)

Para clasificación de texto (spam, sentimientos), **MultinomialNB** es el punto de partida estándar.

Pronóstico con SVM

Máquinas de Soporte Vectorial (MSV / SVM)

Las **Support Vector Machines** son algoritmos versátiles que funcionan tanto para clasificación como para **pronóstico (regresión)**.

- Para regresión se utiliza **SVR** (*Support Vector Regression*)
- Busca un **hiperplano** que mejor se ajuste a los datos dentro de un margen de tolerancia ϵ
- Muy efectivo en espacios de alta dimensionalidad

```
from sklearn.svm import SVR

modelo_svr = SVR(kernel='rbf', C=100, epsilon=0.1)
modelo_svr.fit(X_train, y_train)
y_pred = modelo_svr.predict(X_test)
```

SVM — conceptos clave

 **Hiperplano de decisión** — separa o aproxima los datos en el espacio de características

 **Margen (ϵ)** — en SVR, las predicciones dentro del margen no se penalizan

 **Kernel** — transforma los datos a un espacio donde la separación lineal es posible (`linear` , `rbf` , `poly`)

 **Parámetro C** — controla el equilibrio entre margen amplio y errores de entrenamiento

SVM vs. Redes Neuronales — comparativa rápida

Criterio	SVM / SVR	Red Neuronal (MLP)
Volumen de datos	Funciona bien con pocos datos	Mejora con muchos datos
Interpretabilidad	Media	Baja (caja negra)
Costo computacional	Alto con datos grandes	Escalable con GPU
Ajuste de hiperparámetros	C , <code>kernel</code> , ϵ	Capas, neuronas, tasa de aprendizaje
Uso típico	Señales, series de tiempo	Imágenes, texto, datos complejos

Entrenamiento con datos etiquetados

El flujo de entrenamiento supervisado

1. **Recolectar y etiquetar** los datos — cada ejemplo debe tener su `y` conocida
2. **Dividir** en conjunto de entrenamiento y prueba (`train_test_split`)
3. **Preprocesar** — escalar, codificar, imputar valores faltantes
4. **Entrenar** el modelo con `X_train`, `y_train`
5. **Evaluar** con `X_test`, `y_test` — métricas de desempeño
6. **Ajustar** hiperparámetros si es necesario → repetir

División de datos y preprocesamiento

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Cargar datos
df = pd.read_csv('datos_etiquetados.csv')
X = df.drop(columns='etiqueta')
y = df['etiqueta']

# División 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Escalado (importante para SVM y Redes Neuronales)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```


Evaluación del modelo de clasificación

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = modelo.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Métrica	Qué mide
Accuracy	% de predicciones correctas en total
Precision	De los que predije positivo, ¿cuántos lo eran?
Recall	De los positivos reales, ¿cuántos detecté?
F1-score	Media armónica de Precision y Recall

Matriz de confusión

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	✓ Verdadero Positivo (TP)	✗ Falso Negativo (FN)
Real: Negativo	✗ Falso Positivo (FP)	✓ Verdadero Negativo (TN)

La matriz de confusión revela **dónde falla** el modelo, algo que la accuracy sola no muestra.

Resumen de la semana

Tema	Algoritmo / Herramienta	Tarea principal
Clasificación	<code>MLPClassifier</code> (Red Neuronal)	Predecir categorías
Clasificación	<code>GaussianNB</code> / <code>MultinomialNB</code>	Probabilidades condicionales
Pronóstico	<code>SVR</code> (Máquina de Soporte Vectorial)	Predecir valores continuos
Evaluación	<code>classification_report</code> , <code>confusion_matrix</code>	Medir desempeño del modelo
Preprocesamiento	<code>train_test_split</code> , <code>StandardScaler</code>	Preparar datos para entrenamiento

Para esta semana

- [] Distinguir entre tareas de **clasificación** y **pronóstico** en aprendizaje supervisado
- [] Entrenar un modelo `MLPClassifier` y uno `GaussianNB` sobre un dataset etiquetado
- [] Implementar `SVR` para una tarea de pronóstico numérico
- [] Interpretar la **matriz de confusión** y el `classification_report`
- [] Reflexionar: **¿qué algoritmo conviene más según el tipo y volumen de datos?**

Próxima sesión

K-means clustering · precisión, recall, F1-score, MSE · curvas ROC-AUC · entropía cruzada